

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO1 – limitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	S.Sri Sai Prasanna Durga	Reg. No.:	192424241
Section / Batch:	2024	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Resume Information Extraction using Regular Expressions

Problem Statement:

Develop a Python program using Regular Expressions to extract candidate details such as name, email, mobile number, technical skills, and years of experience from a resume. Generate a summary and check whether the candidate is eligible (minimum 2 years of experience and Python skill).

Algorithm:

1. Import the re module.
2. Store the resume text in a string.
3. Extract the candidate's name using Regular Expressions.
4. Extract email address, mobile number, technical skills, and years of experience.
5. Display the extracted candidate details.
6. Check whether the candidate has at least 2 years of experience and Python skill.
7. Display the eligibility status.

Code:

```
import re
resume = """
Name: Sai Prassana
Email: saiprassana@gmail.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume)
if name:
    name = name.group(1)
emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', resume)
mobiles = re.findall(r'\b[6-9]\d{9}\b', resume)
```

```
skills = ["Python","Java","SQL","Machine Learning","NLP"]
```

```
found_skills = []
```

```
for skill in skills:
```

```
    if re.search(skill, resume, re.IGNORECASE):
```

```
        found_skills.append(skill)
```

```
exp = re.search(r'(\d+)\s+years', resume)
```

```
experience = int(exp.group(1))
```

```
print("----- Candidate Summary -----")
```

```
print("Name:", name)
```

```
print("Email:", emails)
```

```
print("Mobile:", mobiles)
```

```
print("Skills:", found_skills)
```

```
print("Experience:", experience, "Years")
```

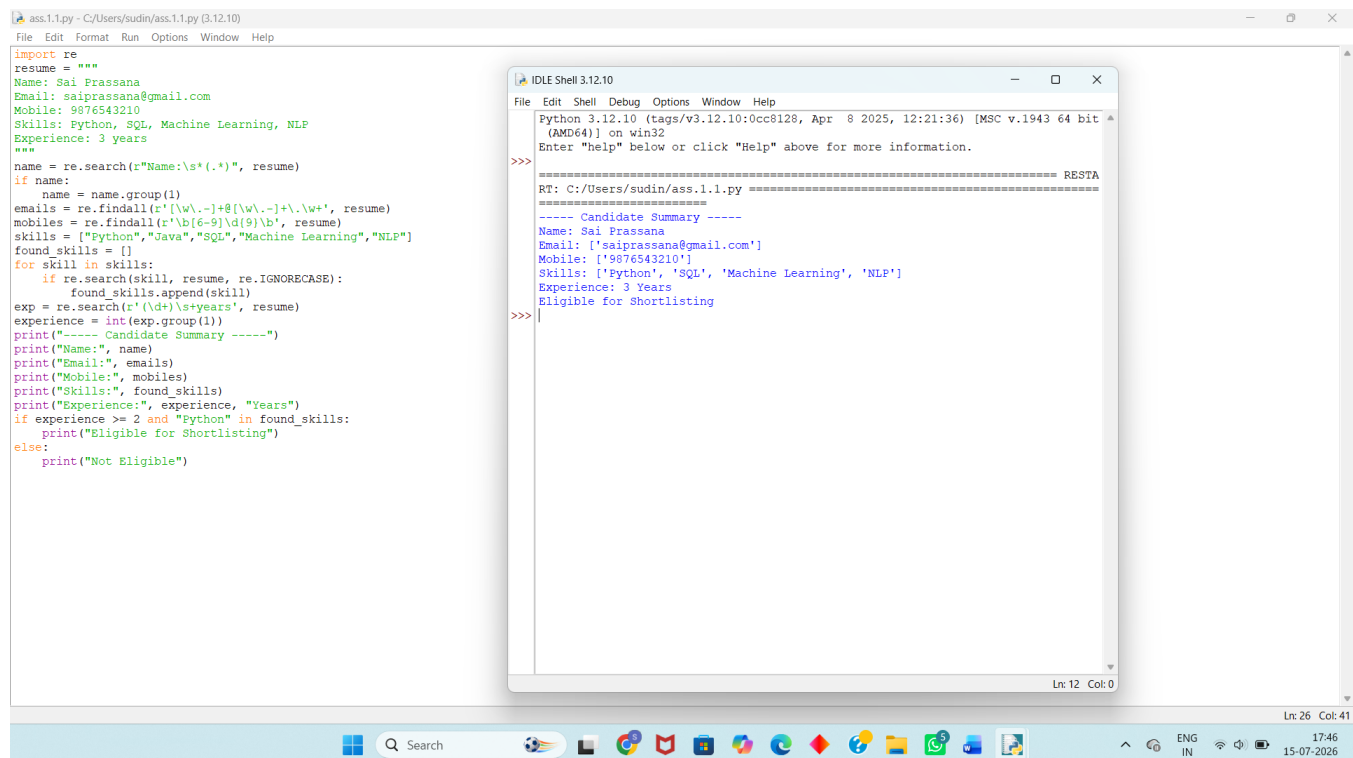
```
if experience >= 2 and "Python" in found_skills:
```

```
    print("Eligible for Shortlisting")
```

```
else:
```

```
    print("Not Eligible")
```

Output:



The screenshot displays a Python script in an IDE window titled 'ass.1.1.py - C:/Users/sudin/ass.1.1.py (3.12.10)'. The script defines a list of skills, searches for them in a resume string, and prints a candidate summary. It also checks if the candidate is eligible for shortlisting based on experience and the presence of 'Python' in their skills.

```
import re
resume = """
Name: Sai Prassana
Email: saiprassana@gmail.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning, NLP
Experience: 3 years
"""
name = re.search(r'Name:\s*(.*)', resume)
if name:
    name = name.group(1)
emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', resume)
mobiles = re.findall(r'\b[6-9]\d{9}\b', resume)
skills = ["Python","Java","SQL","Machine Learning","NLP"]
found_skills = []
for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)
exp = re.search(r'(\d+)\s+years', resume)
experience = int(exp.group(1))
print("----- Candidate Summary -----")
print("Name:", name)
print("Email:", emails)
print("Mobile:", mobiles)
print("Skills:", found_skills)
print("Experience:", experience, "Years")
if experience >= 2 and "Python" in found_skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")
```

The output window shows the execution results, including the candidate summary and the eligibility check result.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTA
RT: C:/Users/sudin/ass.1.1.py
=====

----- Candidate Summary -----
Name: Sai Prassana
Email: ['saiprassana@gmail.com']
Mobile: ['9876543210']
Skills: ['Python', 'SQL', 'Machine Learning', 'NLP']
Experience: 3 Years
Eligible for Shortlisting
```

Q2. Product Search System using Regular Expressions

Problem Statement:

Develop a Python program using Regular Expressions to search products based on exact keyword, prefix, suffix, partial keyword, and case-insensitive matching. Display all matching products and generate a search report.

Algorithm:

1. Import the re module.
2. Store the list of product names.
3. Read the search keyword from the user.
4. Perform exact keyword search.
5. Perform prefix, suffix, partial, and case-insensitive searches.
6. Display all matching product names.
7. Display the total number of matching products for each search.

Code:

```
import re
products = [
    "Laptop",
    "Laptop Bag",
    "Mobile",
    "Mouse",
    "Monitor",
    "Keyboard",
    "Gaming Laptop",
    "Smart Watch"
]
keyword = input("Enter keyword: ")
print("\nExact Match")
exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
```

```

print(exact)
print("\nPrefix Match")
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
print(prefix)
print("\nSuffix Match")
suffix = [p for p in products if re.search(keyword+"$", p, re.IGNORECASE)]
print(suffix)
print("\nPartial Match")
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(partial)
print("\nCase Insensitive Search")
case = partial
print(case)
print("\nReport")
print("Exact:", len(exact))
print("Prefix:", len(prefix))
print("Suffix:", len(suffix))
print("Partial:", len(partial))

```

Output:

The screenshot shows a Python IDE with a file named `ass.1.2.py` and a terminal window displaying the output of the script. The code in the IDE defines a list of products and performs various regex searches based on a user input keyword 'Laptop'.

```

import re
products = [
    "Laptop",
    "Laptop Bag",
    "Mobile",
    "Mouse",
    "Monitor",
    "Keyboard",
    "Gaming Laptop",
    "Smart Watch"
]
keyword = input("Enter keyword: ")
print("\nExact Match")
exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
print(exact)
print("\nPrefix Match")
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
print(prefix)
print("\nSuffix Match")
suffix = [p for p in products if re.search(keyword+"$", p, re.IGNORECASE)]
print(suffix)
print("\nPartial Match")
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(partial)
print("\nCase Insensitive Search")
case = partial
print(case)
print("\nReport")
print("Exact:", len(exact))
print("Prefix:", len(prefix))
print("Suffix:", len(suffix))
print("Partial:", len(partial))

```

The terminal output shows the results of these searches for the keyword 'Laptop':

```

===== RESTART: C:/Users/sudin/ass.1.2.py =====
Enter keyword: Laptop

Exact Match
['Laptop']

Prefix Match
['Laptop', 'Laptop Bag']

Suffix Match
['Laptop', 'Gaming Laptop']

Partial Match
['Laptop', 'Laptop Bag', 'Gaming Laptop']

Case Insensitive Search
['Laptop', 'Laptop Bag', 'Gaming Laptop']

Report
Exact: 1
Prefix: 2
Suffix: 2
Partial: 3

```

Q3. University Registration Validation System

Problem Statement:

Develop a Python program using Regular Expressions to validate student registration details such as register number, institutional email, course code, semester, and mobile number. Display validation messages and the final registration status.

Algorithm:

1. Import the re module.
2. Read the student's register number, email, course code, semester, and mobile number.
3. Validate each field using appropriate Regular Expressions.
4. Display a validation message for each field.
5. Check whether all fields are valid.
6. Display the final registration status.
7. End the program.

Code:

```
import re
reg_no = input("Enter Register Number: ")
email = input("Enter Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
status = True
if re.fullmatch(r'\d{10}', reg_no):
    print("Register Number Valid")
else:
    print("Register Number Invalid")
    status = False
if re.fullmatch(r'[w\.-]+@saveetha\.com', email):
    print("Email Valid")
else:
```



```
print("Email Invalid")
status = False
if re.fullmatch(r'[A-Z]{3}\d{2}', course):
    print("Course Code Valid")
else:
    print("Course Code Invalid")
    status = False
if re.fullmatch(r'[1-8]', semester):
    print("Semester Valid")
else:
    print("Semester Invalid")
    status = False
if re.fullmatch(r'[6-9]\d{9}', mobile):
    print("Mobile Number Valid")
else:
    print("Mobile Number Invalid")
    status = False
print("\nFinal Registration Status")
if status:
    print("Registration Successful")
else:
    print("Registration Failed")
```

Output:

```
ass.1.3.py - C:/Users/sudin/ass.1.3.py (3.12.10)
File Edit Format Run Options Window Help

import re
reg_no = input("Enter Register Number: ")
email = input("Enter Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
status = True
if re.fullmatch(r'\d{10}', reg_no):
    print("Register Number Valid")
else:
    print("Register Number Invalid")
    status = False
if re.fullmatch(r'([a-zA-Z]+@saveetha\.com)', email):
    print("Email Valid")
else:
    print("Email Invalid")
    status = False
if re.fullmatch(r'[A-Z]{3}\d{2}', course):
    print("Course Code Valid")
else:
    print("Course Code Invalid")
    status = False
if re.fullmatch(r'[1-8]', semester):
    print("Semester Valid")
else:
    print("Semester Invalid")
    status = False
if re.fullmatch(r'[6-9]\d{9}', mobile):
    print("Mobile Number Valid")
else:
    print("Mobile Number Invalid")
    status = False
print("\nFinal Registration Status")
if status:
    print("Registration Successful")
else:
    print("Registration Failed")

IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/sudin/ass.1.3.py =====
Enter Register Number: 192424241
Enter Email: 192424241.simats@saveetha.com
Enter Course Code: DSA0307
Enter Semester: 3
Enter Mobile Number: 9581876990
Register Number Invalid
Email Valid
Course Code Invalid
Semester Valid
Mobile Number Valid

Final Registration Status
Registration Failed

>>>
```